

Answering the Expert Questions for Leant2

Sound search, carrier invention, conservative evaluation,
and a concrete implementation plan

Prepared for Vladimir Reshetnikov

September 21, 2026

Scope. An answer to all 31 questions in the nine research areas of *Leant2: Design for Further Improvements*, together with corrections to several premises of those questions. The investigation uses the repository, Lean 4.34.0 source and documentation, and primary research literature.

Repository snapshot.

VladimirReshetnikov/Leant2

784664ff34e819422510a4d470ab07fe48bb736b

What is established here. Mathematical arguments, source-verified integration points, and a reproducible finite-state experiment. Proposed algorithms are distinguished from implemented or measured Leant2 features. The repository's benchmark results were not rerun, and the Lean examples in this report were not compiled.

Main source: docs/proposals/11-further-improvements/.

The question identifiers R1.1–R9.2 retain the ordering of the source document.

Abstract

Leant2’s further-improvements proposal identifies the right obstacles to a practical dependently typed synthesizer, but some of its questions can now be answered more concretely, and several proposed shortcuts require correction. We investigate all 31 expert questions. Important existing ingredients include Canonical-min’s assignment-triggered constraint suspension, Smyth’s higher-order example support, AutoLifter’s synthesis of auxiliary summaries, Hoogle+’s dictionary-based treatment of type classes, and Lean’s reusable induction and library-suggestion interfaces. None is a drop-in solution to Leant2’s complete problem.

We propose a native-Lean constraint orchestrator with persistent branch states, dependency-aware residual evaluation, bounded and backtrackable summary invention, typed recursive-call capabilities, and staged proof obligations. We separate five guarantees that the original plan sometimes conflates: acceptance soundness, pruning soundness, bounded search completeness, ranking optimality, and replay determinism. We prove a safe finite-sample carrier lower-bound criterion, give a minimal four-state example, explain why reverse is already a fold into lists, and show why finite examples cannot generally determine a minimal useful carrier type. The final sections give source-level integration points, an implementation sequence, and an evaluation protocol. Model-guided suggestions remain optional, typed, auditable proposals rather than part of the trusted core.

Contents

1	Executive conclusions and scope	3
1.1	How to read the evidence	4
1.2	Corrections that affect the design	4
2	The five guarantees and a common semantic interface	4
2.1	A shared constraint architecture	5
3	R1: Search control with coupled goals	5
3.1	R1.1. Can independent goals be exploited without copying everything?	5
3.2	R1.2. What admissible heuristic should be used?	6
3.3	R1.3. Can just-in-time learning preserve soundness and stable ranking?	7
3.4	R1.4. Can a wall-clock deadline be machine-independent?	7
4	R2: Carriers, accumulators, and auxiliary state	8
4.1	R2.1. Can finite examples determine a minimal carrier type?	8
4.2	R2.2. What replaces an alphabet for polymorphic functions?	10
4.3	R2.3. Can fusion be run backward to invent auxiliaries?	11
4.4	R2.4. Is there a complete manageable motive grammar for primitive recursion?	12
4.5	R2.5. Can proof-generalization heuristics choose carriers?	12
5	R3: Evaluation of programs with holes	12
5.1	R3.1. Does live bidirectional evaluation already handle higher-order examples?	12
5.2	R3.2. What is a suitable incremental, backtracking normalizer?	13
5.3	R3.3. What is the cheapest provably conservative evaluator?	13
5.4	R3.4. Can top-down angelic evaluation terminate and remain complete?	14
6	R4: Indexed families, motives, and transport	15
6.1	R4.1. Which motive enumeration can be complete and terminating?	15
6.2	R4.2. How should propositional transports be represented and ranked?	15
6.3	R4.3. How aggressively should definitions be unfolded during search?	16
6.4	R4.4. Can Lean’s motive computation be used as a library?	16
7	R5: General recursion from incomplete examples	17
7.1	R5.1. Does a capability interpreter agree with the realized recursive term?	17
7.2	R5.2. Does angelic execution compose with top-down search?	18
7.3	R5.3. Is there a canonical small basis of recursion schemes with known coverage?	18

8	R6: Contracts beyond decidable observations	19
8.1	R6.1. How should value holes and proof holes be interleaved?	19
8.2	R6.2. Which properties are covered by induction, simplification, and arithmetic?	19
8.3	R6.3. What automated induction is usable in Lean now?	20
8.4	R6.4. Is there a principled policy for proof debt?	21
9	R7: Component retrieval at library scale	21
9.1	R7.1. What type abstraction should support dependent reachability?	21
9.2	R7.2. Has type-guided abstraction refinement handled type classes?	22
9.3	R7.3. Can Lean’s suggestion interface accept data goals and examples?	22
10	R8: Ranking and candidate equivalence	23
10.1	R8.1. What is a defensible ranking objective?	23
10.2	R8.2. Which equivalence is cheap and meaningful for dependent terms?	24
11	R9: Learned guidance under a kernel gate	24
11.1	R9.1. How accurate are models at proposing carriers or helpers?	24
11.2	R9.2. How can learned contributions be deterministic and auditable?	25
12	A concrete Lean-native implementation plan	25
12.1	Changes grounded in the inspected source	25
12.2	A state-expansion protocol	26
12.3	Schema invention as a bounded subproblem	27
12.4	Dependency-ordered implementation sequence	27
13	Evaluation, reproducibility, and explicit limits	28
13.1	A benchmark that distinguishes the bottlenecks	28
13.2	Ablations and adversarial regression tests	29
13.3	The included finite experiment	29
13.4	What remains genuinely uncertain	30
14	Conclusion	30
A	Question-by-question coverage map	31
B	Source audit and interpretation of prior work	32
	References	32

1 Executive conclusions and scope

The most useful next step is not to replace Leant2 with a completely new synthesizer. It is to make the existing native-Lean search state explicit, separate its guarantees, and introduce a small number of shared mechanisms: a constraint-dependency service, typed partial observations, and a bounded schema language for carriers, motives, and recursive programs. The difficult invention problems should use those mechanisms rather than each growing a separate search engine.

The source proposal is a design document dated September 18, 2026. Its own measurements refer to commit 33cec8b, Lean 4.34.0, and one Windows workstation. This report instead fixes the inspected repository at commit 784664f; its conclusions about implementation refer to that snapshot, not to hypothetical later changes. Reported baseline successes in the source are evidence about that source’s experiments, not new measurements made here.[1, 2]

Area	Main answer	First implementation consequence
R1	Shared constraints need persistent branch-local state, not a sum of independent goal costs.	Build explicit states and dependency tracking; begin with a safe zero lower bound.
R2	Minimal finite machines are computable; minimal useful Lean carrier types are a different problem.	Use collision-directed, bounded summary synthesis with backtracking.
R3	Higher-order live evaluation already exists; full dependent Lean support does not follow automatically.	Improve kernel-backed per-observation evaluation before adding a certified fragment evaluator.
R4	Useful finite motive classes can be enumerated; unrestricted motive invention cannot be reduced to one finite list.	Reuse Lean’s induction APIs and add targeted, dependency-closed generalization.
R5	A recursive interpreter generally agrees with well-founded realizations propositionally, not by conversion alone.	Preserve equation certificates and prove interpreter–realizer correspondence.
R6	Programs and proofs should exchange constraints, with expensive proving scheduled separately.	Admit local-context proof obligations and exploit recursion-aligned verification conditions.
R7	Hoogle+ already handles type classes through dictionaries.	Extend a typed AND-hypergraph abstraction; use Lean’s actual suggestion interface.
R8	Fixed description cost is defensible, but it is not a theorem about user intent.	Separate immutable ranking from adaptive search; label observational buckets honestly.
R9	No checked accuracy number answers Leant2’s exact carrier-invention question.	Run a controlled study; make replay consume immutable typed proposals rather than rerun a model.

1.1 How to read the evidence

Statements about existing systems below cite their papers or the relevant versioned source. Algorithms introduced with “propose” or “recommend” are designs for Leant2, not claims that an implementation already exists. Mathematical propositions state their assumptions explicitly. A small Python experiment checks finite instances of several arguments; it does not test Lean or establish Leant2 performance. No outside experts were contacted as part of this investigation.

Some literature changes the questions themselves. Smyth’s implementation supports higher-order examples, even though the presentation of its core calculus initially restricts examples. Hoogle+’s implementation explicitly translates type-class constraints into dictionary types. AutoLifter studies precisely the invention of additional summaries needed to make a compositional algorithm work. Canonical-min, published in March 2026, describes suspended constraints that wake when a blocking metavariable is assigned. These are substantive starting points, not merely neighboring research areas.[15, 21, 13, 7]

1.2 Corrections that affect the design

Several corrections deserve emphasis before the detailed answers. A function may be a fold into its result type yet require an expensive step; reverse is an example. A carrier with a final decoding function is more expressive than a direct fold whose state is the required output. A finite set of values observed or generated at one depth is not an upper bound on a Lean type’s cardinality. Summing lower bounds for coupled goals can overestimate the remaining search cost. A final kernel gate cannot recover solutions removed by an incorrect partial-program refuter. Finite agreement on examples is not generally contextual equivalence, and a fixed wall-clock deadline cannot universally provide machine-independent useful results. These are mathematical distinctions, not implementation preferences.

2 The five guarantees and a common semantic interface

Let a query be a local telescope Γ , target type T , contract C , a fixed environment E , and an axiom profile A . Let $\mathcal{G}_b$ be the explicitly declared search grammar under structural bound b . A partial state s contains a term with holes, their well-typed contexts, universe and term constraints, and any residual observations. Write $\text{Comp}(s)$ for its well-typed completions in the stated grammar.

Acceptance soundness. An accepted result contains $p : T$ and, when a contract was requested, a proof of $C(p)$; both are kernel-checked in E and their transitive axiom dependencies satisfy A . This is the role of the existing acceptance gate, which synchronously adds checked declarations in a scratch environment and audits their axioms.[4]

Pruning soundness. A discarded partial state must satisfy

$$\forall p \in \text{Comp}(s), \quad \neg C(p). \tag{1}$$

More generally, a discarded branch can be inconsistent with its typing constraints. This property is independent of acceptance soundness. A bad refuter can make a completely sound accepting synthesizer return no answers.

Bounded completeness. There are two distinct bounded claims. *Enumeration completeness* means that every candidate in a finite declared grammar is eventually considered. *Certification completeness* additionally requires the promised verification procedure to decide the relevant contracts or proof obligations. A tactic timeout does not establish that a candidate is wrong, nor that no proof exists. For general Lean propositions the report recommends explicit “unproved” outcomes rather than pretending that the second guarantee follows from the first.

Ranking optimality. A returned term is optimal only relative to a fixed objective over a precisely specified candidate set, with certified lower bounds for everything still pending. Changing a learned search priority is permissible, but it does not preserve an A* optimality argument for a different, immutable cost function without additional bookkeeping.

Replay determinism. The same inputs, environment, proposal log, and logical work schedule should reproduce the same event sequence and checked outputs. This is not the same as finishing the same number of events before a wall-clock deadline.

A suitable internal result algebra therefore distinguishes a verified candidate, a kernel-checkable refutation, exhaustion of a declared decidable finite search, an unrefuted but unproved candidate, an interrupted search, and an unsupported feature. It should never collapse the last three into “uninhabited.” The default interface can remain simple; the distinctions belong in receipts and diagnostics rather than a proliferation of user settings.

2.1 A shared constraint architecture

A proposed node is conceptually

$$s = (E, \Gamma, \Sigma, p, \mathcal{H}, \mathcal{O}, g, \rho),$$

where Σ is a persistent metavariable/constraint state, $\mathcal{H}$ is the collection of typed holes, $\mathcal{O}$ is the collection of residual observations and proof obligations, g is immutable accumulated cost, and ρ is a stable replay identity. Scheduling scores are separate metadata. A single dependency service answers which assignments may change which constraints, observations, or tactic preconditions.

The first implementation should preserve the current depth-first order as one queue policy. That isolates representation bugs from policy changes. Persistent data structures reduce copying costs, but do not make arbitrary cross-branch mutation safe. The inspected search already uses `Meta.SavedState` around alternatives, while some search bookkeeping is held in references outside that saved state. A first-class node must explicitly classify every such reference as branch-local, immutable shared data, or monotone diagnostic data.[3]

3 R1: Search control with coupled goals

3.1 R1.1. Can independent goals be exploited without copying everything?

Proposed answer. Yes, by sharing immutable structure while representing assignments and dependent continuations explicitly. However, independence must be tested on the full constraint problem, not just on the visible goal expressions. Aesop and Canonical provide important proof-search precedents; the newer Canonical-min makes the assignment-triggered suspension mechanism especially concrete. Its small dependent-type core is not a replacement for Lean’s full metaprogramming state.[5, 6, 7]

Define the support of a goal to include reachable unassigned term and universe metavariables in its type, local declarations and their values, pending constraints, and delayed assignments. Compute this transitively. For example, if one goal mentions $?u$ and another $?v$, but their declarations both depend on $?a$, their visible metavariable sets can be disjoint while the goals remain coupled.

There is a second coupling relation: a contract may relate otherwise independent values. Holes $x : \mathbb{N}$ and $y : \mathbb{N}$ are typing-independent, but the observation $x + y = 7$ couples their acceptable assignments. The dependency graph should therefore have constraint and observation nodes, with hyperedges to all relevant holes. Shared rigid parameters do not by themselves force joint search; shared writable constraints do.

A conservative factorization criterion is disjoint writable support after conditioning on a frozen shared interface. Under that criterion, assignments from the factors commute. Treat an opaque tactic that may inspect or assign arbitrary reachable metavariables as touching their entire support unless it has a narrower declared effect. This sacrifices some parallelism rather than risking cross-branch interference.

For reuse, cache a checked solution *schema* abstracted over a canonical local telescope. Instantiate it freshly and recheck it in the destination state. Do not transplant raw local or metavariable identifiers. For data holes, retain a lazy stream of solutions: reusing only the first inhabitant can lose every solution of the surrounding behavioral contract. A timeout or failed tactic attempt is not a reusable negative theorem. Proven inconsistency or exhaustion of an identical finite constraint problem is.

Concrete recommendation. Add a persistent dependency graph and wakeup queue around native Lean metavariables. Keep branch snapshots for mutable assignments, share syntax and closed lemmas, and factor only components that pass the conservative test. This is a principled middle ground between whole-tree duplication and a permanently fixed sequential order. It does not eliminate the intrinsic combinatorics of genuinely coupled choices.

3.2 R1.2. What admissible heuristic should be used?

Proposed answer. Start with $h(s) = 0$. Introduce stronger lower bounds only with an explicit cost-preserving abstraction argument.

The proposed “one unit per open non-leaf goal” is not generally admissible. An exact local can have zero assigned production cost. Moreover, one unification-producing refinement can close several open constraints. If two constraints each require the same assignment $?n := 0$, summing two unit lower bounds counts the same future action twice. A lower bound on each constraint separately is not automatically additive.

Proposition 3.1 (Abstract-distance lower bound). *Let the concrete search have nonnegative transition costs. Suppose an abstraction maps every concrete solution state to an abstract solution and every concrete transition to an abstract path whose cost is no greater. Then the minimum abstract solution cost from the image of a state is a lower bound on its concrete remaining solution cost.*

Proof. Every concrete solution path maps to an abstract solution path of no larger cost. The minimum over all abstract paths is no larger than the cost of each such mapped path, hence no larger than the minimum concrete solution cost. $\square$

A practical abstraction forgets selected dependent indices and behavioral constraints while retaining a finite typed production hypergraph. Forgetting constraints must add possibilities, not remove them. Zero-cost unification and exact closure must still be possible in the abstraction.

For individually valid lower bounds h_i , $\max_i h_i$ is safe. Their sum requires disjoint work or a proved partition of transition costs among abstractions.

Finite zero-cost closure should be saturated before queue insertion, or the search should impose a positive secondary structural bound. Otherwise infinitely many zero-cost rewrites or casts can prevent fair progress even when the primary cost is nonnegative. The same caution applies to heuristic term-size measures that deliberately ignore proof arguments.

3.3 R1.3. Can just-in-time learning preserve soundness and stable ranking?

Proposed answer. It can preserve acceptance soundness and eventual enumeration while changing the finite-budget answer set. Stable ranking requires a separate, fixed ranking model. Probe establishes a useful precedent for learning from partial successes during enumeration, not a guarantee that user-visible answers remain invariant under such adaptation.[8]

Use two scores. The search score can reward constructions that satisfy more observations or reduce important blockers. The audit score is the fixed objective used for displayed ranking and any optimality certificate. If adaptive scores change, either rekey affected queue entries or explicitly use them only for a separate guidance queue. Reserve a deterministic share of expansions for the unguided grammar queue. This prevents starvation and keeps every bounded alternative reachable, subject to the stated verifier limitations.

Learning updates should occur at deterministic logical epochs rather than elapsed-time thresholds. Record their inputs and resulting parameters. A stable tie-breaker must depend on canonical identifiers, not hash-table iteration order or fresh names derived from time. These choices make a run replayable. They do not make it insensitive to stopping after different numbers of steps.

If an adaptive rule introduces a new carrier type rather than reordering an existing grammar production, it has enlarged the search space. That is sometimes desirable, but it must be recorded as a grammar change rather than justified by a “reordering preserves completeness” argument.

3.4 R1.4. Can a wall-clock deadline be machine-independent?

Proposed answer. Not in general while also requiring useful progress to be returned. Suppose discovering a solution requires k sequential logical steps. A fast machine completes them before the deadline and a slow machine does not. An algorithm that returns the discovered solution on the fast machine cannot always return the same result on the slow one. A constant empty answer would be machine-independent but defeats the intended requirement.

Use deterministic logical work limits for reproducibility and a wall-clock limit as an interrupt boundary. A receipt records the completed event prefix, pending frontier, grammar version, and verified candidates. Deterministic epoch boundaries can stabilize presentation, but the number of completed epochs still depends on speed. Startup calibration is an approximation, not a proof of equivalence: tactic, normalization, and I/O costs vary differently across machines and queries.

Also, the universal Luby restart guarantee concerns a specified Las Vegas restart setting. It does not directly prove optimality for correlated, deterministic Leant2 lanes sharing examples and partial work.[9] A deterministic portfolio with measured quotas is a reasonable engineering choice; its performance should be measured rather than inferred from a nonmatching restart theorem.

4 R2: Carriers, accumulators, and auxiliary state

This area benefits most from separating semantic questions from cost-bounded program construction. Existing fold characterizations and auxiliary-summary synthesis provide more guidance than the original proposal suggests, but neither identifies a unique intended carrier from a few examples.

4.1 R2.1. Can finite examples determine a minimal carrier type?

Proposed answer. There is an exact finite-state problem with a constructive solution, and there is a substantially different Lean program-synthesis problem. A useful implementation should expose the distinction internally.

For lists over A , write a fold realization as

$$q([]) = z, \quad q(a :: xs) = s(a, q(xs)), \quad h(xs) = d(q(xs)), \quad (2)$$

where the carrier is K , $z : K$, $s : A \rightarrow K \rightarrow K$, and $d : K \rightarrow T$ is a finishing or decoding function. A *direct* fold into the result type has $K = T$ and $d = \text{id}$. These are not the same synthesis problem.

There are at least three optimization objectives: the number of reachable semantic states, the syntactic cost of the carrier type, and the total cost of the realized program, its proof, and its execution. They need not agree. For example, $\mathbb{N}$ can encode arbitrarily large finite tuples, but encoding and decoding them can defeat the purpose of introducing a simple product carrier. An optimization over type syntax alone can choose a small type with an enormously expensive step function.

The finite-state problem is decidable

Suppose A has m elements, the allowed output set has r elements, and $K = \text{Fin}(k)$ with $k \geq 1$. There are exactly

$$k k^{mk} r^k = k^{mk+1} r^k \quad (3)$$

choices of initial state, total transition table, and output table. Every finite set of input-output observations can therefore be checked against all k -state machines. Trying $k = 1, 2, \dots$ finds the minimum number of states consistent with the observations.

For a consistent finite sample this search does eventually succeed: build a prefix trie of the *reversed* sample words, add a sink state, and label sample-ending states with their required outputs. Reading reversed words matches the state-transition order of a right fold. Unspecified outputs can be assigned any default. This proves existence of a finite sample-consistent machine, not correctness on unobserved words.

The analogous bounded Lean problem becomes decidable only after specifying an actually finite grammar of carrier types and implementations, finite literal and universe choices where relevant, and a fragment whose typing and observation checks are performed by total decision procedures. Merely bounding the printed size of a carrier while permitting arbitrary helper programs or unresolved proof searches is not such a finite decision problem. A fair enumerator can instead report candidates and unresolved obligations without claiming to decide them all.

A sound lower bound from finite observations

For words u, v , a prefix c is a distinguishing context when the sample contains both $h(c ++ u)$ and $h(c ++ v)$ and their outputs differ. Define the incompatibility graph on selected suffixes by

adding an edge whenever there is such a context.

Proposition 4.1 (Observed-context lower bound). *Every deterministic realization (2) consistent with the sample assigns different states to adjacent suffixes. Consequently the number of reachable states is at least the chromatic number of the incompatibility graph, and in particular at least its clique number.*

Proof. For each prefix c there is a function $S_c : K \rightarrow K$ with $q(c ++ u) = S_c(q(u))$, obtained by composing the fold transitions for the letters of c . If $q(u) = q(v)$, then $d(S_c(q(u))) = d(S_c(q(v)))$. This contradicts any observed distinguishing context. The assignment $u \mapsto q(u)$ is therefore a proper coloring of the graph by reachable states. $\square$

This is a finite-observation consequence of the familiar residual-state view of automata. Angluin’s exact-learning result is relevant, but its query and counterexample oracles provide information that a fixed handful of examples does not.[11]

Two implementation cautions are important. First, equality on the *known* entries of incomplete observation rows is not an equivalence relation. The rows $(0, ?)$, $(?, 0)$, and $(1, 0)$ are pairwise compatible along the first two links but the first and third conflict. Blind union–find merging is therefore unsound. Secondly, the number of colors found by a greedy coloring algorithm is an *upper* bound on chromatic number, not a certified lower bound. A found clique is a safe lower-bound certificate. State merging must also respect transition congruence; a coloring by itself does not construct a valid fold machine.

A minimal four-state example

Let $A = \{0, 1\}$ and let $h(xs)$ mean that both symbols occur in xs . Use suffixes $[], [0], [1], [0, 1]$ and contexts $[], [0], [1]$. Their profiles are as follows; entries give $h(c ++ u)$.

Suffix u	$c = []$	$c = [0]$	$c = [1]$
$[]$	false	false	false
$[0]$	false	false	true
$[1]$	false	true	false
$[0, 1]$	true	true	true

Every pair of rows differs in a column. Thus these observations force a four-clique, and no carrier with fewer than four reachable states works. There are nine distinct input lists among the twelve table entries. A four-state carrier is $\mathbf{Bool} \times \mathbf{Bool}$, recording whether a zero and a one have been seen. Start at $(\text{false}, \text{false})$, set the corresponding flag on each step, and finish by conjunction. Induction on the list proves that each flag has precisely the stated meaning. This gives an all-input implementation and proves that the lower bound is attained.

The accompanying experiment exhausts every one-, two-, and three-state machine and finds no sample-consistent machine. Its counts are 2, 128, and 17,496, respectively, as predicted by (3). It also checks the four-state realization on 2,047 binary words of length at most 10. These tests corroborate the construction; Proposition 4.1 and the flag invariant supply the proofs.

What the test cannot refute

The proposal’s suggestion to count the values of K constructible from the inputs at the current search depth is unsafe as a semantic cardinality argument. A deeper step, a hidden encoding, or

a different literal can reach other values. Such a count can support a *bounded grammar* refutation only after proving that the enumeration exhausts every value relevant to that grammar. It does not show that the type K itself is too small.

Likewise, a direct-fold counterexample does not refute the same type when an arbitrary finish function is allowed. Take $h = \text{tail}$, with $\text{tail}([]) = []$. Then

$$h([]) = h([b]) = [], \quad h([a]) = [] \neq [b] = h([a, b]).$$

A direct fold into lists would have to apply the same step to the two equal states, so it cannot realize tail . Yet with the *same* list carrier, $q = \text{id}$, $s = \text{cons}$, and $d = \text{tail}$ give a valid factorization. A bounded, inexpensive finish grammar is thus part of a meaningful carrier-invention specification.

4.2 R2.2. What replaces an alphabet for polymorphic functions?

Proposed answer. For an explicitly relationally parametric list function, input positions are sufficient; element equality is not automatically observable. With an equality dictionary or additional operations, the model must retain that extra structure.

Assume a pure parametric fragment in which $f_A : \text{List}(A) \rightarrow \text{List}(A)$ satisfies naturality:

$$f_B(\text{map}(g, xs)) = \text{map}(g, f_A(xs)) \quad \text{for every } g : A \rightarrow B. \quad (4)$$

This is the relevant free-theorem assumption.^[12] Let e_n be the list of all positions in $\text{Fin}(n)$ in order. For any list $xs : \text{List}(A)$ of length n , there is a function $g : \text{Fin}(n) \rightarrow A$ mapping positions to their entries, even when entries repeat. Naturality yields

$$f_A(xs) = \text{map}(g, f_{\text{Fin}(n)}(e_n)).$$

Thus $f_{\text{Fin}(n)}(e_n)$ is a word of selected positions describing the result on every length- n input. Using $\text{Fin}(0)$ handles the empty type without assuming an arbitrary function $\mathbb{N} \rightarrow A$ exists. In a fixed higher universe, use the corresponding universe lift of this finite position type.

Proposition 4.2 (Length-bounded canonical testing). *If two functions both satisfy (4) and agree on e_n for each $0 \leq n \leq N$, then they agree on every input of length at most N at every type covered by that naturality assumption.*

Proof. Represent an arbitrary input as $\text{map}(g, e_n)$ and apply naturality to both functions. Their equal position words have equal images under g . $\square$

This is input-length-bounded equality, not equality at every length. It also does not follow merely from seeing a Lean type quantified over Type . For instance, classical decidability permits the noncomputable family that returns xs when A is subsingleton and returns $[]$ otherwise. Mapping a singleton list from a one-element type into a two-element type violates naturality. A syntax and axiom-profile discipline must establish the parametricity premise before exact canonical-test claims are made.

An explicit equality operation permits dependence on equality patterns of positions. Equality-sensitive data-word or register-style models are then more appropriate than a finite alphabet of concrete test values. A comparison or algebraic dictionary carries still more information, and relational reasoning must preserve its operations and assumed laws. In Leant2, use canonical positions as a high-value test heuristic immediately, but permit semantic quotienting only for a fragment with the required theorem.

4.3 R2.3. Can fusion be run backward to invent auxiliaries?

Proposed answer. Yes as a constrained synthesis strategy, but not as a generally complete inverse operation on a few examples. AutoLifter is directly relevant: it synthesizes auxiliary information and combination functions for lifting a reference computation into divide-and-conquer-like schemes. Its minimality property is not global minimization over Lean carrier types, and its discussion explicitly identifies incompleteness caused by greedy choices. Its setting also supplies more structure than arbitrary few-example Lean queries.[13]

The proposed Leant2 variant is *collision-directed summary lifting*. Maintain candidate summary features

$$q(xs) = (\phi_1(xs), \dots, \phi_j(xs)).$$

When observed contexts require two suffixes with equal summaries to be separated, synthesize an additional feature from a bounded typed grammar that distinguishes them. A finite table of candidate features turns feature selection into a weighted covering problem over these collisions. This is proposal generation, not a semantic proof that untested pairs are separated.

Next synthesize a transition that updates all features, and a finishing function. If the transition needs information not retained by the summary, that failure generates another lifting obligation. Crucially, retain alternative feature sets and backtrack: a cheap first summary can be impossible to update within the grammar even though another summary works. The full candidate is certified by the invariant

$$q([]) = z, \quad q(a :: xs) = s(a, q(xs)), \quad C(d \circ q),$$

or, when a reference implementation h is available, by the stronger $d(q(xs)) = h(xs)$ theorem. Learned collisions and feature-selection scores never replace these obligations.

Reverse: a necessary correction and a useful cost example

The fold characterization literature explicitly notes that reverse is a fold.[10] Indeed,

$$\text{reverse}(xs) = \text{foldr}((a, r) \mapsto r ++ [a], [], xs). \quad (5)$$

The base case is immediate, and the induction step is exactly $\text{reverse}(a :: xs) = \text{reverse}(xs) ++ [a]$. So no semantic carrier lower bound can show that reverse needs a carrier other than $\text{List}(A)$.

What fails is an *efficiency or syntax restriction*. Under the usual linked-list cost model, appending to a growing reversed suffix costs $0 + 1 + \dots + (n - 1) = \Theta(n^2)$. A continuation carrier $K = \text{List}(A) \rightarrow \text{List}(A)$ supports the fold

$$z = \text{id}, \quad s(a, k) = \lambda \text{acc}. k(a :: \text{acc}), \quad d(k) = k([]).$$

Its invariant is

$$q(xs)(\text{acc}) = \text{reverse}(xs) ++ \text{acc}. \quad (6)$$

For $a :: xs$, the left side becomes $q(xs)(a :: \text{acc})$; the induction hypothesis and associativity of append give $\text{reverse}(xs) ++ [a] ++ \text{acc} = \text{reverse}(a :: xs) ++ \text{acc}$. The fold builds one closure per element and its final application performs one cons per element, giving linear work under that explicit cost model. This illustrates why carrier invention should optimize the whole schema, not seek a semantic explanation for every shallow-search failure.

For tree flattening, a similar continuation or difference-list summary can avoid repeated copying. For Fibonacci, a pair can retain adjacent values. Such templates belong in a shared bounded schema grammar; their usefulness is demonstrated by invariants and cost models rather than by declaring the unlifted type impossible.

4.4 R2.4. Is there a complete manageable motive grammar for primitive recursion?

Proposed answer. There is a finite, complete enumeration *relative to an explicitly finite template class*. There is no reason to expect one small fixed list to cover arbitrary parameterized primitive-recursive definitions economically.

Begin from the input telescope and the desired result. For each selected recursor, enumerate constant result motives, generalization of eligible parameters into function space, bounded products or dependent pairs of summaries, and bounded abstractions of index expressions. Reversion must include dependencies so that the generalized telescope remains well typed. For an accumulator $a : S$, the motive acquires a factor $S \rightarrow T$; an index-dependent accumulator may instead need a dependent function type. Tupled results and proof-carrying summaries can require Σ -types rather than ordinary products.

A breadth- or cost-bounded traversal of this finite template grammar is complete for the grammar by construction, provided checking and the bounded subsearches have their stated decision guarantees. It is not complete for all useful auxiliaries whose types can be written in Lean. A fair schedule of expanding bounds offers a relative semidecision procedure, not an interactive completeness guarantee. Measure how much each template enlarges branching and how many new benchmark families it unlocks.

4.5 R2.5. Can proof-generalization heuristics choose carriers?

Proposed answer. They transfer as evidence-generating heuristics, not as a complete algorithm. Both problems ask which information must remain available when a recursive or inductive step changes its context.

A failed attempt to use an induction hypothesis can identify an accumulator whose value changes across the call. Generalize that parameter and its necessary dependencies. A failed attempt to compute a next output from the current summary can identify lost information; propose a product component. A repeatedly unresolved equation can suggest an invariant or helper specification. Preserve the actual failed constraint and observed environments so that the proposal is more precise than “try a larger type.”

In the Leant2 architecture, carrier selection, motive generalization, and invariant invention should therefore call one schema service with different views of the same failure. Bound the number and complexity of proposed features; retain an unguided fallback; prove every proposed invariant or helper contract before using it as a permanent pruning fact. An unproved lemma can order exploration, but cannot become an assumption in an accepted proof or justify deleting other solutions.

5 R3: Evaluation of programs with holes

5.1 R3.1. Does live bidirectional evaluation already handle higher-order examples?

Proposed answer. Yes. Smyth’s implemented system supports higher-order examples and polymorphism, and its purpose includes synthesis without trace-complete examples. The source proposal’s blanket first-order description should be corrected. Hazel’s typed hole closures provide part of the operational foundation. Neither fact supplies an already-verified implementation for Lean’s entire dependent calculus.[15, 14]

For Leant2, represent an encountered hole as a typed closure

$$H(m, \sigma) : \tau_m[\sigma],$$

where m was declared in telescope Γ_m and σ is a well-typed substitution from that telescope into the evaluation environment. The same metavariable can be encountered with different values of dependent indices; each closure is typed after its own substitution. Nevertheless all closures must be explained by one assignment to m , not independently chosen outputs.

For a function-valued hole, finite application observations are the natural interface: “at arguments $v_1, \dots, v_j$, this closure must produce these results or satisfy these relations.” This avoids requiring a complete extensional representation of a function. It does not license identifying functions merely because their observed applications agree. Repeated calls on equal arguments must share constraints, and dependent results must be compared at compatible types, with explicit transport when necessary.

The first difficulty in moving to dependent types is therefore not simply that evaluation can encounter a function. It is maintaining typed contextual substitution, index-dependent result constraints, and the correlation of all occurrences of one unknown program. Implementing these on native Lean expressions avoids a second approximation of the user’s type theory.

5.2 R3.2. What is a suitable incremental, backtracking normalizer?

Proposed answer. Reuse the assignment-triggered suspension pattern, but design a branch-aware cache around the actual evaluation contract. Canonical-min is a recent concrete example of dependent constraints suspended on a blocking metavariable and resumed on assignment, with backtracking state management. It is not evidence that a general, certified, hole-assignment cache for Lean 4.34’s kernel is available off the shelf.[7]

The immediate improvement is incremental *observation* evaluation, not a complete replacement normalizer. Each observation records its residual, read dependencies, and status: established true, established false, or unknown. Assigning a hole wakes only affected observations. The existing code instead aggregates blockers across the residual check; this can be refined without changing the evaluator of record.[3]

A cache key must distinguish the expression, local telescope, relevant assignment state, environment, transparency regime, and instance choices. Closed certified results can be shared broadly. Open results should carry branch versions or validated dependency fingerprints. A raw metavariable identifier is insufficient: after backtracking, an identifier or a related name can refer to a different assignment history. In one branch $m = 0$ and in another $m = 1$; a cached evaluation of $m = 0$ must not survive solely because its surface expression is unchanged.

A safe design keeps immutable cache entries with read sets and uses a persistent branch token for validity. Reuse an ancestor entry only when all its recorded dependencies still have the same relevant values. More aggressive dependency minimization is an optimization to verify later. Delayed assignments and universe changes belong in the dependency model, not just ordinary expression-metavariable assignments. Failed evaluation, unsupported reduction, and exhausted fuel remain unknown.

5.3 R3.3. What is the cheapest provably conservative evaluator?

Proposed answer. First retain kernel-backed reduction and make its observations smaller. Then add a small typed, proof-producing fragment evaluator rather than reimplementing the complete kernel.

At query preparation time, decompose recognized conjunctions, finite `List.all` checks, Boolean conjunctions required to be true, and `decide` wrappers using proved equivalences. Preserve arbitrary unrecognized propositions as opaque observations. A custom `BEq` instance need not coincide with propositional equality; rewrite `x == y` into `x = y` only with the appropriate correctness theorem. For a disjunction, retain alternatives: a failed disjunct does not refute the whole observation.

For an initial reflected fragment, choose Boolean and natural-number operations, constructors, projections, and a small collection of list recursors. Give the syntax a denotation into Lean and prove each evaluation rule correct. A reduction result can carry a proof relating the denotation to its residual or value. Unsupported constructs are delegated to Lean or left unknown. Differential tests against kernel reduction are useful for finding bugs, but do not prove a partial evaluator conservative.

The phrase “under-approximate evaluation” requires care. Under-approximating *established reductions* is safe: some true facts remain unknown. Under-approximating the *set of possible outputs* is generally unsafe for refutation: the omitted output might belong to a valid completion. Instead maintain an over-approximation of possible outcomes, or produce a certificate that a false observation is stable under every admissible completion.

Proposition 5.1 (Certified stable residual refutation). *Suppose a well-typed partial term p has a reduction certificate, valid under every admissible assignment to its holes, that an observation $O(p)$ equals false. Then no completion satisfying that observation can extend the state.*

Proof. Instantiate the certificate with any admissible completion. Substitution preserves the certified equality, so the completed observation is false. This contradicts its required truth. $\square$

This argument assumes that the representation and substitution are well typed. In the current source, `instantiatePartial` explicitly permits intermediate expressions whose metavariable local contexts no longer match, for reduction only. That is not a demonstrated bug, but the transformation must be included in the pruning-soundness argument; invoking a correct kernel primitive does not by itself establish that an arbitrary preprocessing transformation preserves all completions.[3]

Lean4Lean is relevant to specifying and checking Lean, but the latest examined revision describes correctness proofs for parts of its typechecker and metatheory, not a ready-made proof of a new Lean 4.34 partial normalizer. Extracting a small evaluator from a separately stated typed fragment is a more contained first verification project.[17]

5.4 R3.4. Can top-down angelic evaluation terminate and remain complete?

Proposed answer. Yes under explicit finite-domain and finite-grammar hypotheses; otherwise only conditional or relative guarantees are justified. Burst provides the important angelic-execution/refinement precedent, but its algorithmic guarantees do not transfer simply by calling another evaluator “angelic.”[16]

An angelic call stands for a set or relation of possible results. A branch may be refuted only if *no* admissible assignment of results can satisfy the observations, using a sound over-approximation of what actual recursive calls can return. Guessing one convenient result and later finding it inconsistent rejects that guess, not every possible completion of the program. Results for repeated identical typed calls must be correlated.

For a finite program grammar and finite call-result domains, retain all consistent symbolic alternatives, refine only by constraints valid for every accepted program, and avoid revisiting the same finite symbolic state. A fair traversal terminates and is complete for those hypotheses

when the checks are total. If result domains are infinite, merely adding fuel does not create that theorem. Fuel exhaustion is unknown, and unbounded fair refinement may be a semidecision strategy rather than a terminating one.

Contract equations such as $f(v) = w$ may constrain a recursive oracle on v when every accepted candidate must satisfy that equation. Arbitrary outputs invented for unobserved calls have a different status. Keep this distinction in the constraint representation so that a convenient synthesis assumption cannot silently become a pruning fact.

6 R4: Indexed families, motives, and transport

6.1 R4.1. Which motive enumeration can be complete and terminating?

Proposed answer. A finite occurrence-abstraction and generalization grammar can be exhaustively searched. Its completeness is relative to that grammar, and it is not automatically closed under all accumulator generalizations.

Fix the recursor, a finite set of eligible maximal index occurrences, a finite collection of context parameters, and a size bound on product, function, and dependent-pair extensions. Enumerate subsets of occurrences to abstract; compute dependency closure for each proposed reversion; form the motive and ask Lean to type-check it. With k eligible occurrences there are at most 2^k raw subsets before typing and symmetry filtering. Adding an unbounded number of auxiliary parameters destroys finiteness, so those extensions must be bounded separately.

The first candidate should be Lean’s ordinary motive, not an expensive powerset search. Next try dependency-closed generalization of indices and parameters actually implicated in a failed induction-hypothesis application. Only then enter the bounded schema grammar of R2. Useful invariants can require new types or relations absent from the target, so abstraction of existing occurrences alone cannot provide unrestricted completeness.

For example, mapping a vector has the motive

$$M(n, xs) = \text{Vec}(B, n).$$

A function threading an accumulator indexed by the remaining length may need $M(n, xs) = \Pi a : S(n), T(n, xs, a)$. This is not merely choosing a subset of occurrences in the original result type. The common schema representation should make the added generalized telescope explicit.

6.2 R4.2. How should propositional transports be represented and ranked?

Proposed answer. Keep native Lean terms and exact types authoritative. Use certified transport simplification as a secondary normalization layer, not an erased setoid language that postpones all typing until the end.

Proof irrelevance identifies proofs of the same proposition. It does not make propositionally equal indices definitionally equal. A term of $\text{Vec}(A, n + m)$ can require transport to inhabit $\text{Vec}(A, m + n)$ even when an arithmetic theorem proves the index equality. The transport endpoints and their dependent family are therefore part of the computational term’s typing information.

Canonicalization can remove reflexive casts, compose adjacent casts, and normalize index expressions using proved equalities. Store the proof needed to justify each replacement. Heterogeneous equality is useful for relating values at different types, but does not remove the obligation to produce a term of the exact requested type. A setoid can be an auxiliary search abstraction

when its simulation and reconstruction obligations are explicit; it is not a general substitute for Lean’s dependent conversion checker.

Giving all casts zero cost can introduce unbounded families of superficially free terms. Use a positive secondary syntax bound, or normalize within a proved finite saturation discipline. Proof arguments may receive a small ranking weight without being ignored for termination of enumeration. Hiding casts in a displayed program is acceptable only when re-elaborating that presentation reconstructs a checked term of the same intended type; otherwise show the cast or an explicit local equality proof.

6.3 R4.3. How aggressively should definitions be unfolded during search?

Proposed answer. Use staged, demand-driven unfolding. Neither full normalization of all states nor a single shallow transparency regime is an adequate general policy. Canonical demonstrates dependent inhabitation with definitional equality; Aesop’s discussion also illustrates the practical limitations of head indexing through definitions. These are precedents, not evidence of a universal optimal transparency schedule.[6, 5]

Begin with reducible weak-head normalization sufficient to expose the target or provider head. On a rigid mismatch, selectively unfold definitions that block a plausible application. Record the transparency level and unfolded head in the cache key. A later, bounded fallback can use broader unfolding rather than permanently treating a shallow mismatch as impossibility.

Normalize index arithmetic with checked lemmas where definitional reduction is insufficient. Do not eagerly normalize a large open recursor merely to compute an index key. Separate a failed attempt at one transparency setting from a proof of non-unifiability. In particular, a search-level policy that chooses not to unfold a definition is not the kernel proving that two expressions cannot be convertible.

The right cost policy is empirical: count successful matches unlocked by additional unfolding, normalization work, and repeated cache misses. Native Lean unification should remain the final check rather than a home-grown approximation of dependent definitional equality.

6.4 R4.4. Can Lean’s motive computation be used as a library?

Proposed answer. Yes, with version-pinned adapters and careful treatment of its preconditions. The inspected Lean 4.34.0 source exposes `Lean.Meta.mkRecursorAppPrefix`, whose documented role is to construct a recursor prefix with the motive obtained by abstracting the major premise and indices. It also exposes `Lean.MVarId.induction`, returning an array of `InductionSubgoal` values with fields and a free-variable substitution.[19]

This gives a concrete integration path instead of implementing a second motive compiler. The same source’s `getMajorTypeIndices` requires indices to be distinct free variables and checks dependency restrictions. A major premise indexed by an expression such as $n + 1$ therefore needs the appropriate higher-level preparation; the low-level API is not a magic solver for arbitrary index expressions.

Wrap each attempt in a complete search transaction. Report structured failure categories such as nonvariable index, inadmissible elimination sort, ill-typed generalized telescope, unresolved instance, or resource limit. Memoize only under a key that includes the target, local context, recursor, proposed generalization, and relevant assignment state. Resource exhaustion must not become a permanent “this motive is impossible” entry.

Start with the high-level induction behavior already available in Lean, then use the lower-level helper when controlled experiments show a benefit. This preserves Lean’s handling of dependent

elimination and keeps maintenance localized when the toolchain version changes.

7 R5: General recursion from incomplete examples

7.1 R5.1. Does a capability interpreter agree with the realized recursive term?

Proposed answer. Agreement must be established by a correspondence theorem. For some structural recursors it can be definitional; for well-founded recursion the reliable general statement is propositional equality using the defining equation and well-founded induction. A search-time interpreter is not certified merely because its syntax resembles a recursive definition.

Represent a recursive problem by a state type S , a result family $B : S \rightarrow \text{Type}$, a well-founded relation $\prec$ on states, and a body with access to a recursive capability

$$\text{rec}_x : (y : S) \rightarrow (y \prec x) \rightarrow B(y).$$

The state must include every recursive argument, including accumulators and any tag needed for mutual recursion. The decreasing relation may depend on only part of the state, but admissibility of every call must be proved for the complete argument tuple.

Proposition 7.1 (Interpreter–realizer agreement). *Suppose two total interpretations of a typed recursive body obey the same recursive equations over a well-founded relation, and the body preserves pointwise equality of its recursive-call results. Then the interpretations agree at every state.*

Proof. Use well-founded induction on x . Every recursive call made by the body at x has an argument $y \prec x$, where the induction hypothesis identifies the two results. The body’s equality-preservation property identifies the result at x . This proves pointwise agreement everywhere. $\square$

For a reflected body language, prove equality preservation by induction on its constructors. If the proof instead uses equality of higher-order recursive oracles, account for the function-extensionality principles used under the selected Lean axiom profile. Do not silently assume that every meta-level interpreter is extensional in the required sense.

A fuelled interpreter should return either a certified value or unknown. Its theorem has the form

$$\text{evalFuel}(p, x, n) = \text{some}(v) \implies \text{realize}(p)(x) = v,$$

not a claim that failure to return a value disproves the contract. A reduction trace can reconstruct this equation from the realized function’s equations. That is often a more controllable certification path than asking kernel normalization to unfold a large well-founded fixpoint directly.

During search, decrease proofs can themselves contain holes. An interpreter must not treat a speculative recursive call as certified merely because its measure appears smaller on one sample. Either establish the call’s required proof, carry it as a guarded obligation, or keep resulting deductions at the heuristic level until the correspondence hypotheses hold.

Implementation recommendation. Introduce a small typed body language only for recursive-call structure, keeping ordinary expressions and types native to Lean. Realize surviving closed bodies through Lean’s recursive definition machinery, retain their equation lemmas, and verify the resulting program and contract proof with the existing gate. The official reference also supplies functional induction principles for supported recursive definitions, which are useful for later contract proofs.[18] Compiled evaluation may rank or test candidates quickly, but a kernel-only refutation policy must reconstruct evidence before deleting a branch on the basis of an untrusted compiled result.

7.2 R5.2. Does angelic execution compose with top-down search?

Proposed answer. Yes, if angelic choices are retained as correlated constraints rather than prematurely committed values. The main loss relative to a bottom-up version-space approach is that top-down refinement may repeatedly rediscover similar residual behavior unless memoization and shared constraints are provided. Burst is the relevant synthesis precedent, not a proof that any top-down adaptation is complete.[16]

In a top-down body, an unknown recursive call produces a fresh symbolic result tied to its typed argument tuple and current function candidate. When two calls have equal arguments, their results must agree. If a contract contains an equation determining that result, constrain it accordingly. If not, derive a relation on possible results by backward propagation. Preserve disjunctions when more than one result can explain the required output.

For example, seeing a residual equation $r_1 + r_2 = 5$ does not justify assigning $r_1 = 0, r_2 = 5$ and discarding the body if that assignment later fails. That is only one explanation. Moreover, if the two symbols represent the same recursive call, the residual must include $r_1 = r_2$. Ignoring that correlation admits spurious angelic explanations and can make refinement ineffective.

A practical hybrid uses top-down native typing to choose the recursion scheme and body skeleton, with bottom-up enumeration of small closed components in observed environments. The component store may group terms by observed behavior as an index, while retaining alternatives when future contexts can distinguish them. Eventually execute the complete candidate without angelic choices and discharge the full contract. Unknown recursive values help steer synthesis; they do not become new axioms.

7.3 R5.3. Is there a canonical small basis of recursion schemes with known coverage?

Proposed answer. No verified measurement found in this investigation establishes such a basis for Leant2’s language, trust policy, and benchmarks. Scheme families are useful engineering choices, but “canonical” and “covers textbook programs” are not precise enough to imply a theorem or a percentage.

Use an ordered collection with separately reported coverage: ordinary structural recursion including `Nat`; structural recursion with parameters or tupled results; continuation-style accumulators; course-of-values recursion; and well-founded recursion with a bounded grammar of measures and lexicographic combinations. Add mutual and nested cases through explicit Lean-compatible adapters when corresponding benchmark families justify the cost. One general well-founded scheme may be semantically expressive, yet far worse for search than several specialized schemes.

Fibonacci illustrates why scheme and carrier should not be evaluated independently: course-of-values recursion realizes one presentation, while ordinary natural-number recursion with a pair realizes another. List zip can be expressed through recursion on one list with the other as a generalized parameter. The best search representation is not determined by the surface number of recursive arguments.

Evaluate the marginal gain of each scheme on family-held-out tasks, with both known-carrier and invented-carrier tracks. Publish failures by cause: no expression in the current grammar, body search interrupted, termination proof unresolved, behavioral counterexample, or contract proof unresolved. Without this separation, a scheme can appear to lack expressiveness when the actual bottleneck is a missing library component or proof lemma.

8 R6: Contracts beyond decidable observations

8.1 R6.1. How should value holes and proof holes be interleaved?

Proposed answer. Treat both as constraints, but distinguish cheap propagation from expensive proof search. Waiting for every program hole to close is too late; running arbitrary tactics on every partial state is too expensive.

The proposition that proof search is meaningless until its program holes are filled is too strong. A proof goal such as $?n = 3$ can determine $?n$ by reflexivity and unification. A constructor proof or a subtype invariant can similarly restrict an implementation choice. The proper restriction is that a tactic attempt must preserve all shared assignments transactionally and should run only when its expected benefit justifies the work.

Use compositional verification conditions for recognized constructors, recursors, and certified components. Cheap operations—unification, constructor decomposition, reflexive equality, and small reflected arithmetic—run eagerly. More expensive simplification, induction, and library search are separate obligations with dependency watchers. Synquid’s refinement-based decomposition is a useful conceptual precedent, but Lean propositions are richer than the decidable refinement fragment of that system.[20]

A complete local proof pattern: length-preserving map

For $f : A \rightarrow B$, synthesize

$$F(xs) : \{ys : \text{List}(B) \mid \text{length}(ys) = \text{length}(xs)\}.$$

The nil branch returns $([], \text{refl})$. In the cons branch, suppose the recursive result is (ys, h) with $h : \text{length}(ys) = \text{length}(xs)$. Return

$$(f(a) :: ys, \text{congrArg}(\text{Nat.succ}, h)),$$

after the definitional reductions of list length. The proof obligations are local and follow exactly the data construction. Projecting the first component yields a map function with a length-preservation proof.

This pattern avoids asking a generic tactic to discover the entire induction about an anonymous recursor after synthesis has finished. It also provides constraints during construction: choosing a branch output of the wrong length creates a local obligation that can often be refuted immediately.

A concrete prerequisite is visible in the repository: the current `proofPortfolio` returns without trying tactics when the target contains free variables. It cannot simply be reused unchanged for these local branch obligations, which intentionally contain variables and an induction hypothesis. Introduce a separate local-context proof service with explicit scope and transactional semantics.[3]

A Lean-only analogue of finite liquid-style abstraction can start from a bounded set of proposition templates involving lengths, membership, ordering, and selected index equations. Infer and test candidate invariants, then prove the selected ones with Lean. Without a decision procedure for the chosen templates, this remains an incomplete proof-search discipline, not a new complete refinement checker.

8.2 R6.2. Which properties are covered by induction, simplification, and arithmetic?

Proposed answer. Define a sufficient, certificate-producing class rather than promise a complete syntactic characterization of all theorems that a tactic might prove.

Fix a structural recursion scheme, a proposed invariant P , an oriented finite set of computation and rewrite lemmas, and a decidable residual logic. A property belongs to the proposed automation class when every branch verification condition, after applying the recursive hypotheses and the specified rewrites, reduces to a formula in that residual logic and its decision procedure proves it. This membership test is operational and certificate-producing. It need not predict success without constructing the verification conditions.

Proposition 8.1 (Recursion-aligned verification). *Suppose each constructor branch of a structurally recursive program proves its postcondition from the postconditions of all permitted recursive calls. Then structural induction proves the postcondition for every input.*

Proof. Apply the datatype’s induction principle. Its hypotheses supply exactly the postconditions assumed by the corresponding certified branch. Applying that branch proof establishes the induction conclusion. $\square$

Length preservation, affine size bounds, and many constructor-level invariants fit a first implementation when their residual arithmetic is Presburger arithmetic. Properties involving permutations, nested uses such as $f(f(xs)) = xs$, or nonlinear arithmetic may require auxiliary lemmas or a different invariant. They are not automatically impossible; they are outside the initial guaranteed tactic recipe until additional certified rules are provided.

Keep a provenance record of the recursion scheme and the equations used. It makes branch-condition generation cheap and avoids reverse-engineering an arbitrary kernel term. Successful helper lemmas should be abstracted over their contexts and stored as checked reusable components, subject to the same trust profile as ordinary providers.

8.3 R6.3. What automated induction is usable in Lean now?

Proposed answer. Lean 4.34 provides ordinary induction and the more targeted `fun_induction` and `fun_cases` facilities. The reference describes functional induction for supported non-mutually-recursive structural or well-founded definitions, including generalization controls. These are concrete building blocks already missing from the current small portfolio. They do not constitute autonomous discovery of every needed induction or lemma.[18]

Combine a bounded choice of induction variable or synthesized function with simplification by its equations, the local induction hypotheses, and `omega` or other checked closers. `grind` is useful general-purpose proof automation, but it should not be described as an automatic replacement for choosing all induction principles and inventing strengthening lemmas. Optional Aesop-style rule search and Canonical-style inhabitation supply additional approaches.[5, 6]

A raw recursor expression does not necessarily have the named functional induction theorem expected by a high-level tactic. Preserve the original recursive equations and realize promising programs as definitions when using functional induction, or apply the relevant structural induction principle directly. This is another reason to retain synthesis provenance.

The current Lean Hammer work combines premise selection with external reasoning and checked proof reconstruction. It is relevant to a later optional proof lane, not a core-only, dependency-free induction solution. Its measurements concern theorem proving, not automatic certification of Leant2’s invented recursive programs.[23]

Use Lean resource limits and interrupt-aware wrappers for every tactic call. A library interface with a fuel or heartbeat limit provides controlled work, not necessarily a perfectly predictable wall-clock duration. Failed or interrupted attempts leave the candidate unproved unless a separate counterexample or contradiction has been certified.

8.4 R6.4. Is there a principled policy for proof debt?

Proposed answer. There are useful decision-theoretic heuristics, but the coupled, state-changing proof obligations here do not automatically satisfy the assumptions of a classical independent-arm bandit theorem.

For comparison, suppose two independent candidate tests have fixed costs c_1, c_2 and rejection probabilities p_1, p_2 , and testing stops at the first rejection. Testing 1 then 2 costs in expectation $c_1 + (1 - p_1)c_2$; the reverse order costs $c_2 + (1 - p_2)c_1$. The first order is better exactly when $p_1/c_1 \geq p_2/c_2$. This justifies a simple rejection-per-cost heuristic for suitable tests. Proof attempts differ: success may discharge several coupled obligations, failure is often inconclusive, and both outcomes can change subsequent search.

Use a deterministic tiered policy. Eager propagation pays very small local obligations. A bounded medium tier attempts simplification and arithmetic when their data dependencies stabilize. An expensive tier considers induction and retrieval after the candidate has survived known counterexamples and its recursive structure is stable. Reserve a fixed share of work for older pending obligations so that promising but difficult states are not starved.

Limit *active* proof debt by suspending states, not by declaring them false. Charge proof work to the same logical ledger as construction and evaluation. Learn cost estimates for ordering if useful, while keeping the fairness rule and immutable ranking objective unchanged. Before acceptance, every required proof debt must be discharged. The exact thresholds are benchmark parameters to calibrate for a release, not quantities already known to be optimal.

9 R7: Component retrieval at library scale

9.1 R7.1. What type abstraction should support dependent reachability?

Proposed answer. Use a sound typed AND-hypergraph abstraction with refinement, not only a graph of result-type heads. Preserve a native-Lean checking boundary. Hooghe’s type-guided abstraction refinement is the closest directly useful precedent, but Lean adds genuinely dependent indices, universe constraints, and a richer instance mechanism.[21]

A component with several required arguments is an AND-transition: all its preconditions must be realizable with compatible substitutions. A simple path from some available type head to the goal head can ignore an impossible second argument. It can also forget that the same type variable occurs in several positions. Therefore abstract nodes should retain normalized head structure, a bounded pattern of shared type variables, selected index predicates, universe relationships, and explicit dictionary requirements. The abstraction may forget additional constraints as long as it *over-approximates* concrete applicability.

Lean’s ordinary local variables are reusable. If a Petri-net representation is adopted, it must represent contraction and weakening appropriately; consuming an input token linearly without those rules would discard valid programs. Likewise, the abstract representation of a dependent component must include every concrete instantiation promised by the retrieval policy. Otherwise abstract unreachability is not a sound reason to omit it.

When a concrete path fails Lean type checking, inspect the mismatch and add a useful predicate to the abstraction: an index equality, a relationship between repeated type parameters, or a dictionary requirement. Merely checking each proposed term by unification is not itself abstraction *refinement*; the next abstract search must actually use the newly learned distinction.

A path-length cutoff of three can be an effective fast lane. It is an incomplete heuristic unless the declared search grammar itself has that composition bound. Maintain a fair broader fallback,

or state the cutoff in any exhaustion result. Use indexing to shortlist plausible components; final native typing decides whether an actual application is legal.

9.2 R7.2. Has type-guided abstraction refinement handled type classes?

Proposed answer. Yes. Hoogle+’s implementation translates type classes into explicit dictionary types: class operations consume dictionaries, and instances construct them. Its discussion also identifies limitations, including unsupported higher-kinded type variables such as the parameter in `Monad m`. Thus the starting point is an existing construction to adapt, not an entirely unexplored type-class problem.^[21]

The corresponding Lean representation should use actual instance arguments. An instance constraint is a typed obligation, not merely a Boolean label saying that a class name occurs. Resolving it may assign metavariables, depend on local instances, or fail because parameters are still unknown. Run speculative instance search inside the branch transaction and suspend underdetermined requests rather than caching them as permanently impossible.

A cache key should include the full normalized class application, universe instantiation, relevant local context and local instances, environment fingerprint, and assignment dependencies. The proposal’s shorthand “(class, type)” key is insufficient for classes with several parameters or dependent arguments. Instance dictionaries can also affect behavior: two valid structures on the same carrier type may interpret operations differently. Behavioral observations and compiled-evaluation caches must retain which dictionary was chosen.

Implement dictionary-aware filtering before ambitious dependent refinement. Measure whether it reduces the number of failed component applications without causing missed successes under the declared abstraction. Gradually retain more index and universe information only where traces show that coarse abstraction generates excessive spurious paths.

9.3 R7.3. Can Lean’s suggestion interface accept data goals and examples?

Proposed answer. Its type accepts an arbitrary goal metavariable, and its configuration already has an optional textual hint. That is a useful integration point, but it is not a guarantee that an installed selector understands data synthesis or examples.

In the inspected Lean 4.34.0 source,

```
abbrev Selector := MVarId -> Config -> MetaM (Array Suggestion)
-- Selected Config fields:
maxSuggestions : Nat
caller          : Option String
filter          : Name -> MetaM Bool
hint            : Option String
```

The source explicitly says that Lean does not provide a default library suggestion engine. The hint is arbitrary and a selector may ignore it. There is no required structured input–output–example field in this interface.^[22]

A Leant2 adapter can pass a data goal, identify itself through the caller field, and serialize a compact observation summary into the hint for a selector designed to consume it. For a reliable local default, use the native provider index and abstract component graph directly. A specialized selector can add typed behavioral features without changing the Lean interface, but exact application checking remains necessary downstream.

A training corpus can be built from suitable Std or Mathlib definitions: retain their type and approved context, hide the body, extract used components as labels, and generate well-typed

observations when the function is executable. This is a dataset proposal, not an existing measured Leant2 corpus. Hide aliases, generated equations, and directly exposing helper declarations where they would leak the answer. Split by module, chronology, and near-duplicate algorithm family, not merely by randomly selected definitions. Keep instance choices and the environment snapshot fixed. General lemmas may remain available when the benchmark is explicitly about realistic library use, but disclose that availability.

The Lean Hammer paper describes retrieval over theorem-proving states and library signatures. Its strong premise-selection results do not establish relevance accuracy for data goals constrained by examples.[23] Use a separate held-out evaluation for the latter. Network-backed selectors must remain an explicit optional extension; a core-only default should not silently send user goals or environment contents to a service.

10 R8: Ranking and candidate equivalence

10.1 R8.1. What is a defensible ranking objective?

Proposed answer. Minimum description cost under a fixed grammar is defensible as a stated prior, not as a theorem that the first answer is the user’s intended program. A useful objective can combine code, proof, execution, and presentation costs:

$$J(p, \pi) = -\log P_\theta(p) + \lambda_\pi \text{proofCost}(\pi) + \lambda_t \text{runCost}(p) + \lambda_h \text{presentationCost}(p). \quad (7)$$

For optimality claims, fix θ , the weights, the cost definitions, and the candidate grammar for the run. An estimated runtime term is a ranking feature; it is not a proved complexity bound unless separately certified. Probabilistic grammars and learned enumeration costs are established ideas, but their adaptation to user-preferred Lean inhabitants requires measurement.[8]

A grammar distribution is often over derivations, not programs. Multiple derivations of the same term can distort both probability and size. Normalize construction paths or define program cost as the minimum cost of its permitted derivations. The implementation must match the chosen mathematical objective before claiming that a cost-ordered search is optimal.

Ignoring an explicit argument is not always suspicious: constants and projections are frequently correct. Input-use penalties should therefore be soft, contract-aware features rather than a semantic rejection rule. Similarly, syntactic occurrence is only an over-approximation of influence; $x - x$ mentions x without depending on it extensionally. Exact information flow is not obtained merely by counting variable occurrences.

Proposition 10.1 (A valid stopping certificate). *For a fixed objective J , suppose every unexplored or uncertified candidate is covered by a frontier state with a valid lower bound $L(s)$ on its final cost. If a certified candidate has cost J^* and $J^* \leq \inf_s L(s)$, then no covered candidate has smaller cost.*

Proof. Every covered candidate has cost at least its state’s lower bound, which is at least J^* . Exhausted parts of the search contain no unexamined candidate. $\square$

The frontier must include deferred proof obligations, suspended carrier choices, and candidates discovered by any auxiliary lane. Adaptive search priorities are not these lower bounds. A margin-based stopping rule gives only the corresponding approximation guarantee; it does not establish semantic uniqueness or intendedness.

Show a small diverse set of certified candidates, their important behavioral differences, and a distinguishing input when one is found. For a ranking benchmark, compare reference rank and

family-level alternatives, but do not treat one reference implementation as the unique acceptable program. Human preference claims require an actual user study rather than inference from shorter syntax.

10.2 R8.2. Which equivalence is cheap and meaningful for dependent terms?

Proposed answer. Use a hierarchy and name the level achieved. No single cheap test serves kernel equality, extensional equality, and useful behavioral diversity.

First canonicalize bound-variable names and approved syntactic normal forms. Then use Lean definitional equality when economical. A separately checked propositional equality can establish more identifications; function equality or heterogeneous equality requires its corresponding proof and trust assumptions. Observational agreement on generated inputs is a useful final *bucket*, but is not generally an equality proof.

For dependent terms, retain exact types and any transport witnesses. Proof irrelevance can collapse proofs of the same proposition; it does not collapse arbitrary values at propositionally related types. Type formers such as `fun _ => Empty` and `fun _ => Unit` may have similar syntactic shapes while being meaningfully different. Grouping them for presentation must not be called semantic deduplication.

Finite observational quotienting is safe for destructive search pruning only when it is a congruence for every future context admitted by the search and all relevant environments. Agreement on the current examples rarely gives that guarantee for open, higher-order, or dependent terms. Otherwise retain multiple representatives lazily and split a bucket when a new observation separates them. The elementary counterexample is the identity function and the constant-zero function: they agree on input zero and differ on input one.

Contract inputs alone often cannot separate any accepted candidates, because all candidates already satisfy them. Add fresh well-typed probes and, in a proved parametric fragment, the canonical position inputs from R2.2. Report “same on these probes” unless a stronger equality certificate is available. Printed strings should be presentation artifacts, not the primary identity of terms or constraints.

11 R9: Learned guidance under a kernel gate

11.1 R9.1. How accurate are models at proposing carriers or helpers?

Proposed answer. The inspected primary literature does not provide a verified number for the exact question: Lean types plus sparse examples, invention of a useful carrier or helper, successful realization and proof, and a short end-to-end interactive budget. Theorem-proving and premise-retrieval results cannot fill this gap by extrapolation.[23, 6] This is a measurement question for which a focused experiment is more useful than an unsupported percentage.

Separate proposal quality into stages. Does the suggested type elaborate? Does it admit a body in the same bounded grammar used by the baseline? Does the realized program satisfy the observations? Is the requested full contract proved? Does the entire run, including model inference, meet the budget? Report all stages, not just syntactically plausible suggestions.

Use paired comparisons against the same grammar with no model, a fixed handwritten carrier list, and collision-directed summary invention. Evaluate both known-carrier and invented-carrier tracks. Keep the model’s permitted components, examples, and axiom profile identical to those of the baseline. Report top-*k* proposal success and end-to-end verified success separately, together with elaboration cost, proof cost, inference latency, hardware, and resource use. A model that

proposes the right carrier but consumes the entire interaction budget has not achieved the requested end-to-end result.

Hold out algorithm families and close variants to reduce leakage from textbook solutions. A carrier is not wrong merely because it is presented by an isomorphic type, but acceptable equivalence and cost criteria must be defined in advance. Count a proposed helper as useful only after its implementation and required proof are obtained, or report it explicitly as an unresolved sketch. Preserve failed proposals as well as successful ones in the experimental record.

The best first insertion point is one bounded batch of schema proposals at query preparation or a deterministic decision epoch, not a network call inside every node expansion. The report recommends this as an experiment, not as evidence that an optional model is already a net performance win.

11.2 R9.2. How can learned contributions be deterministic and auditable?

Proposed answer. Make search replay consume immutable, typed proposal records. Temperature zero and a fixed random seed alone are not a sufficient specification of byte-for-byte reproducible inference across implementations and hardware. Recording exact proposals separates proof replay from model regeneration.

A receipt should identify the environment and toolchain, target and contract, local telescope, grammar version, axiom profile, fixed ranking objective, logical schedule, proposal bytes and order, checked terms, and certificates. If model regeneration is itself a research objective, also identify model, tokenizer, inference implementation, and relevant runtime configuration. These are provenance requirements, not additional logical axioms. A signature or digest protects provenance integrity; the kernel check establishes the meaning of a program and its proof.

There is an important security boundary beyond logical soundness. A model must not supply arbitrary Lean commands, elaborator extensions, or tactics for execution inside the trusted process. Such code can have effects before the final kernel gate runs. Accept a restricted typed abstract syntax or strictly parsed term proposal over approved constants, and isolate any untrusted elaboration that remains. The kernel gate does not undo I/O or other effects produced by executing a proposal generator.

If learned suggestions only reorder a fixed grammar, the enumeration argument remains applicable with a fair baseline lane. If they introduce new carriers or helpers outside it, create an explicit grammar-expansion epoch and update the meaning of exhaustion and ranking certificates. Never claim both “the model changes only order” and “the model can introduce arbitrary new types” without reconciling those two interfaces.

Finally, replaying an accepted proof establishes acceptance, not the absence of a better candidate or the correctness of every prior pruning action. Those stronger claims require the relevant search frontier and pruning certificates. Keep the default implementation entirely model-free; optional proposals can improve discovery without becoming part of the logical trust base or making the ordinary core depend on service availability.

12 A concrete Lean-native implementation plan

12.1 Changes grounded in the inspected source

The following table distinguishes integration points that exist from proposed new services. Paths without a “proposed” qualification are present in the inspected snapshot. This is a source-based plan, not a patch claimed to have been built or benchmarked.[1, 3, 4]

Location or boundary	Recommended change and its reason
<code>Leant2/Search/Core.lean</code> : alternatives and obligations	Extract a first-class state and queue policy while preserving the current depth-first order for regression comparison. Inventory every mutable reference for rollback behavior.
<code>Core.lean</code> : residual predicates and blockers	Give each observation its own status, dependencies, and branch-valid residual cache. Keep kernel-backed checking as the initial evaluator.
<code>Core.lean</code> : proof portfolio	Add a separate transactional proof service for local contexts. Its goals may contain locals and induction hypotheses, unlike the current closed-goal portfolio.
<code>Leant2/Accept/Gate.lean</code>	Retain synchronous kernel checking and axiom auditing. Make the binding between the requested contract and the supplied proof type explicit in the acceptance API.
Proposed constraint service	Track transitive expression and universe dependencies, delayed assignments, typed observation closures, and wakeups.
Proposed schema service	Enumerate bounded carriers, generalized motives, summary features, and recursive-call schemes through one typed interface.
Lean 4.34 induction adapter	Reuse <code>MVarId.induction</code> and <code>mkRecursorAppPrefix</code> ; preserve returned substitutions and provenance.
Lean suggestion adapter	Consume the <code>Selector</code> API or the local provider graph; include examples only through a selector that explicitly understands them.
Proposed receipt/replay service	Record immutable query, grammar, schedule, proposal, certificate, and trust-profile data. Distinguish an interrupted prefix from exhausted search.

The gate currently checks a supplied proof against its supplied proof type. With a larger proposal ecosystem, require the boundary to verify that this type is the requested $C(p)$, or a specified equivalent query result type, for the exact accepted program. This is interface hardening suggested by the function signature, not a reproduced erroneous acceptance in the current frontend. A proof of an unrelated true proposition must not be relabeled as a proof of the user’s contract.[4]

12.2 A state-expansion protocol

The following is algorithmic pseudocode, not a compilable Lean API sketch. It describes ordering and certificate boundaries rather than new trusted reduction rules.

```

expand(state):
    restore_complete_branch_state(state)
    saturate_bounded_cheap_typing_constraints()
    wake_observations_whose_dependencies_changed()

    for observation in ready_observations:
        result := evaluate_with_evidence_or_unknown(observation)
        if result contains a stable_refutation_certificate:
            record_refutation_and_stop_this_branch()

    if all_program_and_proof_holes_are_closed:
        check_exact_query_binding()
        return acceptance_gate(program, contract_proof, profile)

    if a local proof obligation is due under the fixed work policy:
        run_bounded_transactional_proof_service()
    else:
        decision := choose_ready_constraint_or_schema_hole()
        proposals := fair_merge(grammar_choices, recorded_guidance)
        for proposal in proposals:
            child := apply_in_fresh_transaction(proposal)
            if child is well_typed_or_has_valid_pending_constraints:
                enqueue(child, immutable_cost, adaptive_priority)

```

Every unknown branch remains eligible under the declared fair schedule unless it is explicitly outside the current finite grammar. Observation and proof services return typed outcomes, not a Boolean that conflates failure, unsupported syntax, resource exhaustion, and refutation. This discipline is more important than whether the first queue implementation is a heap, stack, or a deterministic portfolio of both.

12.3 Schema invention as a bounded subproblem

A schema proposal should contain its carrier or generalized telescope, seed, step holes, finishing hole, recursive-call permissions, and invariant holes. For a fold, the input and output interface is

$$(A, T, C, b) \mapsto (K, z, s, d, \mathcal{V}),$$

where $\mathcal{V}$ contains the typing, behavioral, and invariant obligations needed to certify the schema. A carrier suggestion without a realizable step and finish is an intermediate hypothesis, not a successful synthesis.

Store a version-space of bounded feature proposals rather than committing to the first collision separator. If one feature can be computed on whole inputs but cannot be updated compositionally in the current step grammar, try the next feature set. Helper invention must obey the same rule: a helper with an underdetermined observation table remains a synthesis problem until its implementation and required properties have been discharged.

This common interface permits a small default library of useful schemas, collision-directed suggestions, and optional recorded model proposals to coexist. It also makes ablations meaningful: all proposal sources feed the same realization, evaluation, and certification pipeline.

12.4 Dependency-ordered implementation sequence

Phase A: preserve behavior while exposing guarantees. Refactor state management without changing the old enumeration order. Add structured outcomes, a logical work ledger, exact query binding at acceptance, and deterministic receipts. Test sibling-branch rollback

and replay before changing heuristics. This phase establishes the conditions under which later measurements mean what they claim.

Phase B: obtain more information from the existing evaluator. Split recognized observations through checked rules, track dependencies per observation, and add counterexample reuse. Do not first implement an entire replacement kernel. Measure avoided reevaluations and the number of observations that remain opaque or unsupported.

Phase C: add high-value native reasoning. Permit local-context proof obligations, integrate source-verified induction adapters, and add demand-driven arithmetic and provider lookup. Introduce a small number of typed branch and recursion schemas with explicit bounds. This addresses gaps that do not require a breakthrough in carrier invention.

Phase D: bounded summary and motive invention. Add backtrackable feature grammars, sample incompatibility certificates, and schema-specific invariants. Implement the carrier-given track first so that failures of body synthesis are not misattributed to invention. Add true abstraction refinement to data-component retrieval when spurious paths become a measured bottleneck.

Phase E: certified fragment evaluation and richer recursion. Only after residual traces identify the expensive fragment, implement its proof-producing evaluator. Add capability-based course-of-values or well-founded recursion with an explicit interpreter–realizer theorem and retained equations. Unsupported cases continue through slower native paths or remain unknown, never false by default.

Phase F: learning and preference experiments. With reproducible traces and stable baselines in place, compare learned proposals and ranking policies. Keep model-free behavior available and avoid turning one optional optimization into a prerequisite for ordinary synthesis. These phases have logical dependencies, not promised delivery durations. The order differs from a plan that treats carrier invention as the prerequisite for every other gain. Correct local proof handling, per-observation evaluation, and native indexed induction can improve the system before a general carrier inventor is available.

13 Evaluation, reproducibility, and explicit limits

13.1 A benchmark that distinguishes the bottlenecks

Retain the existing regression suites, but add tasks partitioned by the capability needed: conditionals and arithmetic; ordinary and accumulating recursion; multiple recursive results; indexed families and transport; universal contracts; large-library composition; and underdetermined behavioral specifications. Record the allowed providers, trust profile, recursion schemes, type grammar, and example sets for every task.

Use paired tracks for carrier given versus invented, reference implementation available versus examples only, and trace-complete versus incomplete examples. For dependent tasks, include hand-checked target motives as diagnostic data, without giving them to the invention track. A supplied motive that still fails isolates body search, transport, or proof automation as the obstacle.

The main performance outcome is the number of *verified* solutions at specified wall-clock budgets, accompanied by logical work counts. Report unrefuted/unproved candidates separately. Also report time to first verified candidate, verification cost, peak retained frontier, residual evaluations, cache validity checks, and the rank of an appropriate reference solution. Avoid counting a finite-test survivor as a universally correct program.

Evaluation should include complete-cost accounting. A retrieval lane must include index construction or explicitly report amortized warm-cache results. A model lane includes inference and elaboration. A compiled evaluator lane includes whatever evidence reconstruction is required by its trust policy. Do not compare a kernel-only baseline with an unverified execution path as if their guarantees were identical.

13.2 Ablations and adversarial regression tests

The most informative ablations disable one mechanism at a time: per-observation invalidation, dependency-aware ordering, carrier lifting, local proof obligations, indexed induction, abstract retrieval refinement, and learned proposals. Use identical query snapshots and deterministic logical budgets for algorithmic comparisons; use repeated paired runs for wall-clock effects.

Add adversarial cases specifically for the new invariants. Backtrack after a metavariable assignment and revisit an identically printed residual. Introduce local instances with different behavior. Couple two typing-independent data holes by one observation. Use an intentionally non-lawful Boolean equality instance. Interrupt a proof attempt that would eventually succeed. Offer two terms equal on training inputs but different on a fresh one. Propose an ill-scoped dependent closure. Introduce a speculative recursive call whose decrease proof is still open. Every case should have an expected outcome class, not just an expected printed candidate.

For ranking certificates, put the eventual cheaper program in a deferred carrier or proof lane. The engine must not stop with an optimality claim while that lane has a smaller valid lower bound. For learned guidance, include a proposal that lies outside the declared grammar and verify that its admission changes the grammar epoch rather than silently invalidating an exhaustion certificate.

13.3 The included finite experiment

The archive contains a dependency-free Python script and its JSON output. Its role is to make several small counterexamples and finite counts reproducible. It is not a prototype of the Leant2 search engine.

Check	Result
Distinct observations for the four-state lower bound	9
One-state machines exhaustively rejected	2
Two-state machines exhaustively rejected	128
Three-state machines exhaustively rejected	17,496
Binary inputs checked against the four-state realization	2,047
Ternary inputs checked for the direct reverse fold	3,280

The script additionally checks elementary witnesses for nontransitive partial-row compatibility, the invalid shared-goal cost sum, finite-probe nonequivalence, branch-blind cache invalidity, and transitive dependency coupling. These are small mathematical regression examples, not evidence

that the corresponding bugs have occurred in Leant2. The reverse equality and the minimal four-state claim have proofs in Section 4; finite testing is not substituted for those proofs.

Run the experiment from the archive directory with

```
python3 experiments/check_claims.py --output experiments/results.json
```

No Lean compiler was available in the execution environment for this report. Consequently no Lean snippet was compiled, no proposed tactic adapter was executed, and no repository benchmark was independently reproduced. The API claims come from reading the pinned source and versioned documentation. The algorithmic pseudocode is deliberately not presented as tested Lean code.

13.4 What remains genuinely uncertain

The largest empirical uncertainty is not whether more carrier expressions can be enumerated, but whether collision-driven lifting sufficiently reduces the joint search for features, steps, finishes, and proofs on realistic queries. The second is whether typed residual reuse remains cheaper than reduction after accounting for dependency bookkeeping and branching. The third is whether generalization and local verification conditions cover a large useful fraction of dependent programs without uncontrolled branching.

Model accuracy and human ranking preferences remain unmeasured for the exact Leant2 workload. A strong result in theorem proving or examples-only synthesis is evidence of a technique’s potential, not a substitute for those measurements. None of these uncertainties prevents implementing the bounded algorithms and testing the hypotheses. They instead determine which claims should be made as design proposals and which require new experiments.

14 Conclusion

The expert questions do not all have the same status. Some have concrete answers in existing work or current Lean APIs: higher-order live examples, dictionary-aware type-guided abstraction, assignment-triggered constraints, and reusable motive construction. Some have precise restricted solutions: minimal sample-consistent finite machines, bounded motive enumeration, recursion-aligned proof obligations, and abstract-distance search lower bounds. Others are empirical design questions, especially efficient carrier invention, proof scheduling, learned suggestions, and user-preferred ranking.

The strongest unifying recommendation is to preserve native Lean typing and make uncertainty and evidence explicit at every boundary. A sound gate, a conservative partial evaluator, a fair bounded enumerator, an immutable ranking objective, and a deterministic replay mechanism solve different problems. Conflating them creates attractive but invalid shortcuts.

A practical next version of Leant2 can make substantial progress without assuming a general solution to dependent program synthesis. Start with explicit states, observation-level dependencies, local proof obligations, and Lean’s own induction machinery. Then add bounded, backtrackable summary invention and certified recursive realization. This creates a measurable path from the current synthesizer to stronger programs and contracts, while retaining a small and intelligible logical trust boundary.

A Question-by-question coverage map

Every enumerated question in the nine source research areas has an explicit answer. R8 and R9 share the source file `12-ranking.tex`; they are separate areas in this report. The source’s consolidated expert table is a priority summary, not an additional set of questions.[2]

ID	Subject of the answer	Page
R1.1	Shared metavariables, independence, and cross-branch memoization	5
R1.2	Admissible lower bounds with coupled goals	6
R1.3	Just-in-time learning and ranking guarantees	7
R1.4	Wall-clock portability and deterministic replay	7
R2.1	Minimal carriers from finite examples	8
R2.2	Polymorphic alphabets and canonical position inputs	10
R2.3	Backward fusion and auxiliary-summary invention	11
R2.4	Motives for primitive recursion with parameters	12
R2.5	Generalization and lemma-invention heuristics	12
R3.1	Higher-order and dependent live evaluation	12
R3.2	Incremental normalization and rollback invalidation	13
R3.3	A conservative evaluator and the role of Lean4Lean	13
R3.4	Angelic top-down termination and completeness	14
R4.1	Finite motive enumeration and generalization closure	15
R4.2	Canonical transport handling	15
R4.3	Definitional unfolding policies	16
R4.4	Lean motive-computation library interfaces	16
R5.1	Interpreter–recursor correspondence and evaluation	17
R5.2	Angelic execution in top-down synthesis	18
R5.3	A small recursion basis and measured coverage	18
R6.1	Interleaving implementation and proof holes	19
R6.2	A certifiable class of recursion-aligned properties	19
R6.3	Current Lean induction and proof automation	20
R6.4	Scheduling proof debt	21
R7.1	Dependent type abstraction for component reachability	21
R7.2	Type classes in type-guided abstraction refinement	22
R7.3	Data-oriented premise selection and training data	22
R8.1	Formal ranking objectives and stopping certificates	23
R8.2	Meaningful candidate equivalence	24
R9.1	Accuracy of learned carrier and helper proposals	24
R9.2	Deterministic and auditable learned guidance	25

B Source audit and interpretation of prior work

Source	What was used, and what was not inferred
Leant2 proposal and code	All nine question areas and the expert summary; inspected construction, residual, proof-portfolio, and acceptance code. No independent performance reproduction.
Canonical-min (2026)	Assignment-triggered suspension and a small dependent inhabitation/unification implementation. Not a claim of drop-in full Lean compatibility.
Smyth (2020)	Higher-order examples in the implementation, live bidirectional evaluation, and removal of trace-completeness requirements. Not a certified evaluator for Lean’s dependent kernel.
AutoLifter (2024 revision)	Auxiliary-summary and combinator synthesis; its own restrictions on minimality and greedy search. Not minimal Lean carrier types inferred uniquely from arbitrary finite samples.
Hoogle+ (2020)	Type-guided abstraction refinement and explicit dictionary encoding. Not unrestricted support for Lean dependency, universes, or all higher-kinded classes.
Lean 4.34 source/manual	Exact induction helpers, functional induction facilities, and the suggestion interface including its hint field and lack of a built-in selector engine.
Lean4Lean (2025 revision)	An external Lean typechecker and verification of parts of the checker/metatheory. Not a complete proof for every rule of a newly proposed partial evaluator.
Lean Hammer (2026 revision)	Current premise-selection and proof-reconstruction architecture. Not an empirical estimate of carrier invention or data-synthesis accuracy.

References

- [1] Vladimir Reshetnikov. *Leant2 repository and README*. Inspected snapshot 784664ff34e819422510a4d470ab07fe48bb736b, September 18, 2026. <https://github.com/VladimirReshetnikov/Leant2/tree/784664ff34e819422510a4d470ab07fe48bb736b>.
- [2] Leant2 project. *Leant2: Design for Further Improvements*. September 18, 2026. Sections R1–R9 and the consolidated expert-question table; same pinned snapshot as [1]. The proposal’s original measurements refer to an earlier commit and are not reproduced in this report. <https://github.com/VladimirReshetnikov/Leant2/tree/784664ff34e819422510a4d470ab07fe48bb736b/docs/proposals/11-further-improvements>.
- [3] Leant2 project. *Construction search*, `Leant2/Search/Core.lean`. Pinned snapshot as [1]; especially `alternative`, `instantiatePartial`, `mkResidual`, `kernelDecide`, `evalResidual`, `partialRefute`, and `proofPortfolio`. <https://github.com/VladimirReshetnikov/Leant2/blob/784664ff34e819422510a4d470ab07fe48bb736b/Leant2/Search/Core.lean>.
- [4] Leant2 project. *The acceptance gate*, `Leant2/Accept/Gate.lean`. Pinned snapshot as [1]. <https://github.com/VladimirReshetnikov/Leant2/blob/784664ff34e819422510a4d470ab07fe48bb736b/Leant2/Accept/Gate.lean>.
- [5] Jannis Limperg and Asta Halkjær From. Aesop: White-Box Best-First Proof Search for Lean. *CPP*, 2023. DOI: 10.1145/3573105.3575671. Author manuscript: <https://people.compute.dtu.dk/ahfrom/aesop-camera-ready.pdf>.
- [6] Chase Norman and Jeremy Avigad. *Canonical for Automated Theorem Proving in Lean*. 2025. arXiv:2504.06239. <https://arxiv.org/abs/2504.06239>.

- [7] Chase Norman and Jeremy Avigad. *Implementing Dependent Type Theory Inhabitation and Unification*. March 2, 2026. arXiv:2603.01463. <https://arxiv.org/html/2603.01463v1>.
- [8] Shraddha Barke, Hila Peleg, and Nadia Polikarpova. Just-in-Time Learning for Bottom-Up Enumerative Synthesis. *Proceedings of the ACM on Programming Languages*, 4(OOPSLA), 2020. DOI: 10.1145/3428295. <https://arxiv.org/abs/2010.08663>.
- [9] Michael Luby, Alistair Sinclair, and David Zuckerman. Optimal Speedup of Las Vegas Algorithms. *Information Processing Letters*, 47:173–180, 1993. DOI: 10.1016/0020-0190(93)90029-9.
- [10] Jeremy Gibbons, Graham Hutton, and Thorsten Altenkirch. When is a Function a Fold or an Unfold? *Electronic Notes in Theoretical Computer Science*, 44(1), 2001. Author publication record: <https://www.cs.ox.ac.uk/publications/publication2368-abstract.html>. Related author slides used for the explicit reverse example: <https://people.cs.nott.ac.uk/pszgmh/wgp01/gibbons-slides.pdf>.
- [11] Dana Angluin. Learning Regular Sets from Queries and Counterexamples. *Information and Computation*, 75(2):87–106, 1987. DOI: 10.1016/0890-5401(87)90052-6.
- [12] Philip Wadler. Theorems for Free! *Functional Programming Languages and Computer Architecture*, 1989. DOI: 10.1145/99370.99404.
- [13] Ruyi Ji, Yuwei Zhao, Yingfei Xiong, Di Wang, Lu Zhang, and Zhenjiang Hu. *Decomposition-Based Synthesis for Applying Divide-and-Conquer-Like Algorithmic Paradigms*. arXiv:2202.12193, revision 4, January 29, 2024; accepted for ACM TOPLAS as noted by that revision. Especially the algorithm, minimality discussion, and limitations. <https://arxiv.org/html/2202.12193v4>.
- [14] Cyrus Omar, Ian Voysey, Ravi Chugh, and Matthew A. Hammer. Live Functional Programming with Typed Holes. *Proceedings of the ACM on Programming Languages*, 3(POPL), 2019. DOI: 10.1145/3290327. <https://arxiv.org/html/1805.00155v4>.
- [15] Justin Lubin, Nick Collins, Cyrus Omar, and Ravi Chugh. Program Sketching with Live Bidirectional Evaluation. *Proceedings of the ACM on Programming Languages*, 4(ICFP), article 109, 2020. DOI: 10.1145/3408991. The implementation discussion includes higher-order examples and polymorphism. <https://arxiv.org/abs/1911.00583>.
- [16] Anders Miltner, Adrian Trejo Nuñez, Ana Brendel, Swarat Chaudhuri, and Isil Dillig. Bottom-up Synthesis of Recursive Functional Programs using Angelic Execution. *Proceedings of the ACM on Programming Languages*, 6(POPL), 2022. DOI: 10.1145/3498682. <https://arxiv.org/abs/2107.06253>.
- [17] Mario Carneiro. *Lean4Lean: Verifying a Typechecker for Lean*, in *Lean*. arXiv:2403.14064, revision 3, September 14, 2025. <https://arxiv.org/abs/2403.14064>.
- [18] Lean development team. *The Lean Language Reference*, version 4.34.0. Especially the tactic reference entries for induction, functional induction, simplification, arithmetic, and general proof automation. <https://lean-lang.org/doc/reference/4.34.0/Tactic-Proofs/Tactic-Reference/>.
- [19] Lean development team. *Induction metaprogramming implementation*, version 4.34.0. `src/Lean/Meta/Tactic/Induction.lean`. Includes `getMajorTypeIndices`, `mkRecursorAppPrefix`, and `Lean.MVarId.induction`. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/Meta/Tactic/Induction.lean>.
- [20] Nadia Polikarpova, Ivan Kuraj, and Armando Solar-Lezama. Program Synthesis from Polymorphic Refinement Types. *PLDI*, 2016. DOI: 10.1145/2908080.2908093. <https://arxiv.org/abs/1510.08419>.
- [21] Zheng Guo, Michael James, David Justo, Jiaxiao Zhou, Ziteng Wang, Ranjit Jhala, and Nadia Polikarpova. Program Synthesis by Type-Guided Abstraction Refinement. *Proceedings of the ACM on Programming Languages*, 4(POPL), article 12, 2020. DOI: 10.1145/3371080. Especially Section 5.1 on type classes and implementation limitations. https://ranjitjhala.github.io/static/hoogle_plus.pdf.
- [22] Lean development team. *An API for library suggestion algorithms*, version 4.34.0. `src/Lean/LibrarySuggestions/Basic.lean`. Includes `Selector`, `Config.hint`, and the documented absence of a default library suggestion engine. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/LibrarySuggestions/Basic.lean>.
- [23] Thomas Zhu, Joshua Clune, Jeremy Avigad, Albert Q. Jiang, and Sean Welleck. *Premise Selection for a Lean Hammer*. ICLR 2026; arXiv:2506.07477, revision 2, February 25, 2026. <https://arxiv.org/html/2506.07477v2>.